

Relative path: detection/model.py
Filename: model.py Extension: .py

```
"""
OASIS v2 – Layer 1: Detection Model wrapper
=====
Combines four signals into anomaly_score:
1. GRU reconstruction error – catches sequence-shaped anomalies.
2. Baseline profiler rarity_score – catches statistical anomalies.
3. Cold-start rule engine – used only when entity has too little history.
4. geo_velocity_flag (Layer 0) – dedicated impossible-travel signal, combined
   via max() so it can't be diluted by the other two, same principle as the
   cold-start rule fallback.
5. cumulative drift snapshot signal – catches long-horizon low_and_slow
   buildup that the continuously-adapting EWMA is intentionally designed to
   absorb for legitimate insider drift.
"""
from __future__ import annotations

from datetime import datetime
from typing import Dict, List

import numpy as np

from detection.gru_autoencoder import GRUAutoencoder
from profiling.baseline_model import BaselineProfiler, FEATURE_NAMES, extract_features


def cold_start_rule_score(session: dict):
    triggered = []
    score = 0.0
    if session["auth_failures"] >= 5:
        score += 0.4
        triggered.append("auth_failures")
    hour = datetime.fromisoformat(session["start_time"]).hour
    if hour < 5 or hour > 22:
        score += 0.2
        triggered.append("off_hours")
    if session["is_new_device"]:
        score += 0.2
        triggered.append("device_fingerprint")
    if len(set(session["geo_locations"])) > 1:
        score += 0.2
        triggered.append("geo_locations")
    return min(1.0, score), triggered


class DetectionLayer:
    def __init__(self, cold_start_min_sessions: int = 5, embed_dim: int = 8,
                 hidden_dim: int = 16, max_len: int = 15, seed: int = 42):
        self.cold_start_min_sessions = cold_start_min_sessions
        self.embed_dim = embed_dim
        self.hidden_dim = hidden_dim
        self.max_len = max_len
        self.seed = seed
        self.vocab: Dict[str, int] = {}
        self.gru: GRUAutoencoder | None = None
        self.profiler = BaselineProfiler(cold_start_min_sessions=cold_start_min_sessions,
                                         decay_half_life_sessions=8)
        self._normal_recon_errors: List[float] = []

    def _to_ids(self, session: dict) -> List[int]:
        return [self.vocab[a] for a in session["command_sequence"] if a in self.vocab]

    def fit(self, normal_sessions: List[dict], epochs: int = 4, lr: float = 0.02, verbose=True):
        vocab_terms = sorted({a for s in normal_sessions for a in s["command_sequence"]})
        self.vocab = {a: i for i, a in enumerate(vocab_terms)}
        self.gru = GRUAutoencoder(vocab_size=len(self.vocab), embed_dim=self.embed_dim,
                                   hidden_dim=self.hidden_dim, max_len=self.max_len,
                                   seed=self.seed)
        rng = np.random.default_rng(self.seed)
        order = list(range(len(normal_sessions)))
        for epoch in range(epochs):
            rng.shuffle(order)
            total = 0.0
            for i in order:
```

```

        total += self.gru.train_step(self._to_ids(normal_sessions[i]), lr=lr)
    if verbose:
        print(f"    [detection] epoch {epoch+1}/{epochs}    avg recon loss = {total/len(normal_sessions):.4f}")

    self._normal_recon_errors = [
        self.gru.reconstruction_error(self._to_ids(s)) for s in normal_sessions
    ]
    self._err_mean = float(np.mean(self._normal_recon_errors))
    self._err_std = float(np.std(self._normal_recon_errors)) + 1e-6

def _normalize_recon(self, err: float) -> float:
    z = (err - self._err_mean) / self._err_std
    return float(np.clip(1 - np.exp(-max(z, 0) / 3.0), 0.0, 1.0))

def score_all(self, sessions_in_time_order: List[dict]) -> List[dict]:
    results = []
    for session in sessions_in_time_order:
        rarity_score, is_cold_start, deviation_order, geo_velocity_flag, drift_score = self.profiler.score(session)

        ids = self._to_ids(session)
        recon_err, per_step_err = self.gru.reconstruction_error(ids, per_step=True)
        recon_norm = self._normalize_recon(recon_err)
        geo_velocity_score = 1.0 if geo_velocity_flag else 0.0

        if is_cold_start:
            rule_score, triggered = cold_start_rule_score(session)
            anomaly_score = max(rule_score, rarity_score, geo_velocity_score, drift_score)
            used_fallback = True
            contributing = triggered + deviation_order[:2]
        else:
            anomaly_score = max(recon_norm, rarity_score, geo_velocity_score, drift_score)
            used_fallback = False
            contributing = list(deviation_order[:3])
            if per_step_err and recon_norm >= rarity_score and recon_norm >= geo_velocity_score and recon_norm >= drift_score:
                worst_t = int(np.argmax(per_step_err))
                if worst_t < len(session["command_sequence"]):
                    contributing.insert(0, f"action:{session['command_sequence'][worst_t]}")

        if geo_velocity_flag:
            contributing.insert(0, "impossible_travel_velocity")
        if drift_score >= max(recon_norm, rarity_score, geo_velocity_score, 0.45) and "low_and_slow_drift" not in contributing:
            contributing.insert(0, "low_and_slow_drift")

        results.append({
            "session_id": session["session_id"],
            "entity_id": session["entity_id"],
            "anomaly_score": round(float(anomaly_score), 4),
            "rarity_score": round(float(rarity_score), 4),
            "is_cold_start": is_cold_start,
            "used_fallback_rule": used_fallback,
            "geo_velocity_flag": geo_velocity_flag,
            "drift_score": round(float(drift_score), 4),
            "contributing_features": contributing[:5],
        })
    return results

```